Deep Reinforcement Learning-Based Deterministic Routing and Scheduling for Mixed-Criticality Flows

Hao Yu, Tarik Taleb, *Senior Member, IEEE*, and Jiawei Zhang

***Abstract*—Deterministic networking has recently drawn much attention by investigating deterministic flow scheduling. Combined with artificial intelligent (AI) technologies, it can be leveraged as a promising network technology for facilitating automated network configuration in the Industrial Internet of Things (IIoT). However, the stricter requirements of the IIoT have posed significant challenges, that is, deterministic and bounded latency for time-critical applications. This article incorporates deep reinforcement learning (DRL) in cycle specified queuing and forwarding and proposes a DRL-based deterministic flow scheduler (Deep-DFS) to solve the deterministic flow routing and scheduling problem. Novel delay aware network representations, action masking and criticality aware reward function design are proposed to make deep-DFS more scalable and efficient. Simulation experiments are conducted to evaluate the performances of deep-DFS, and the results show that deep-DFS can schedule more flows than the other benchmark methods (heuristic- and AI-based methods).**

***Index Terms*—6G and artificial intelligence, deep reinforcement learning (DRL), deterministic networking (DetNet), Industrial Internet of Things (IIoT), Internet of Things (IoT), mixed-criticality flows.**

I. Introduction

THE Industrial Internet of Things (IIoT) adopted in manufacturing and factory automation is typically implemented by a specialized network for data exchange among sensors, actuators, and other production equipment. To facilitate information exchange, industrial networks have evolved over the years and are expected to satisfy the emerging and challenging requirements of the new operation contexts [1]. On the one hand, conventional network technologies could not provide deterministic and efficient communications for the industrial needs. To support critical data flows generated by IIoT applications with bounded low latency and low jitter, the IEEE time-sensitive networking (TSN) and the IETF deterministic networking (DetNet) work group have been initiated to study timing guarantee for critical traffic. On the other hand, with the huge amount of the ever-increasing IIoT connectivity, the network administrators need to rely on humans to design, configure, and manage sophisticated and dynamic industrial scenarios, which is not efficient and sustainable. The next-generation network automation represented by artificial intelligence (AI)-based technologies is proposed to tackle this challenge. Along with the advent of the network programmability of 5G networks, the AI-enabled paradigm will carry out the intelligent automated network configuration, optimization, and management in the Industry 5.0 era.

Recently, the IETF DetNet (DN) working group has been studying deterministic data transmission by incorporating segment routing (SR) in layer 3 to extend TSN technologies for queuing and scheduling, in order to support deterministic bounded latency and jitter for time-critical traffic [2]. In particular, regarding the strength of programmability of SR, the working group is currently specifying a cycle specified queuing and forwarding (CSQF) mechanism [3] to schedule the flows in a more flexible and scalable way, where the forwarding time slot can be specified for the packets, which will increase the bandwidth efficiency. In CSQF, multiple queues in the output port open in a round-robin way and transmission cycles repeat periodically at each port. By defining the SR identifiers in IP packets, it can determine the packet routing and forwarding at each hop, specifically, deciding the routing and transmission time slots along the selected path for all packets of time-critical flows, so that the end-to-end (E2E) latency is controlled in a deterministic way. We refer to it as the deterministic flow routing and scheduling (DFRS) problem in this article. To this end, a network controller is required to collect the overall network information for deciding the proper scheduling for flows.

Different kinds of solutions have been proposed to solve the flow scheduling problem: solver-based methods, e.g., an integer linear programming (ILP) tool [4] or heuristic-based methods,

Manuscript received 6 April 2022; revised 10 July 2022 and 6 October 2022; accepted 29 October 2022. Date of publication 15 November 2022; date of current version 13 July 2023. This work was supported in part by the European Unions Horizon 2020 Research and Innovation Program through the Charity and Accordion projects under Grant 101016509 and Grant 871793, in part by the Academy of Finland 6Genesis project under Grant 318927, and in part by the Academy of Finland IDEA-MILL Project under Grant 352428. Paper no. TII-22-1478. *(Corresponding author: Hao Yu.)*

Hao Yu and Tarik Taleb are with the Center of Wireless Communications, The University of Oulu, 90570 Oulu, Finland (e-mail: hao.yu@oulu.fi; tarik.taleb@oulu.fi).

Jiawei Zhang is with the State Key Laboratory of Information Photonics and Optical Communications, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: zjw@bupt.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TII.2022.3222314.

Digital Object Identifier 10.1109/TII.2022.3222314

e.g., list-based methods [5]. Nevertheless, due to the fact that the high computational complexity of solver-based methods and heuristic-based methods are usually handcrafted with certain expertise, scalable, and intelligent scheduling approaches are desired to solve the flow scheduling problem. Therefore, Zhong et al. [6] leveraged deep reinforcement learning (DRL) to solve the time-triggered (TT) flow scheduling problem incrementally. However, the agent trained in [6] was only used to make routing decisions, the transmission cycles for time-critical packets were determined by a heuristic-based method, which would choose the earliest time slots along the path for packet forwarding to minimize the E2E delay of the TT flow. Nevertheless, the users of time sensitive networks only care about the delay bound guarantee, and any earlier delivery of any particular packet is not necessary. In addition, it will cause network congestion to always choose earliest available time slots to minimize the E2E delays without considering different deadlines of flows.

In this article, we will investigate on the routing and scheduling problem of mixed-criticality DN flows for deterministic performance guarantee and propose a DRL-based deterministic routing and scheduling approach to solve this problem. Note that minimizing E2E delay is not the objective of this article, the proposed approaches schedule the flows with the derived E2E delays near the deadlines without exhausting network resources. In addition, we will also consider the multiple criticalities of the timing requirements of DN flows, i.e., hard real-time (HRT), soft real-time (SRT), and train the DRL agent to make the decision on flow routing and scheduling with the objective of maximizing the number of HRT flows scheduled and the utilities of SRT flows. The contributions of this article are shown as follows.

1) A DRL-based deterministic flow scheduler (Deep-DFS) is devised based on a Markov decision process (MDP) approach for the flow routing and scheduling problem, and a branching dueling Q-network (BDQN) is introduced into the framework to derive the optimal policy of the MDP model.
2) We propose the following methods to enhance the efficiency and schedulability of deep-DFS:
    1) we divide a complete flow schedule into multiple simple actions to increase the scheduling scalability;
    2) we use a latency aware network representation method to better extract key information for flow routing and scheduling;
    3) we introduce action masking to filter invalid actions to avoid too many negative rewards;
    4) we design a criticality aware reward function to schedule the flows with different criticalities.
3) Finally, an extensive performance evaluation is carried out with both single path and multipath scheduling. The results show that, in an incremental scenario, the deep-DFS scheme can schedule more DN flows than other benchmark methods and multipath scheduling has better performance than single path scheduling.

The rest of this article is organized as follows. We introduce some related work in Section II. The system model and problem formulation are presented in Section III. Section IV illustrates the details of BDQN-based DFRS methods. The evaluation of the proposed method is discussed in Section V. Finally, Section VI concludes this article.

II. Related Work

In the context of TSN, most studies in the literature focus on the static scheduling and dynamic reconfiguration of gate openings and closings at output ports to satisfy a certain traffic matrix. In this case, routing information is generally given by the spanning tree protocol operating at layer 2. For 802.1Qbv, the disadvantages of the flow-based time-aware shaper (TAS) are limitation of the gate control list (GCL) synthesis solution space and the long time it takes to solve the GCL synthesis in the case of large-scale networks. To solve this problem, Hellmanns et al. [7] proposed a stream-based, class-based TAS without perflow scheduling, which relaxes the assumption that gate openings for multiple ST queues are enforced to be mutually exclusive. Furthermore, Barzegaran et al. [8] proposed a more general flexible window-based scheduling model, i.e., besides the abovementioned constraint relaxation, windows do not have to be aligned and can be placed at any time slot on nodes in networks. For network reconfiguration under dynamically changing requirements, the concept of multistream gate control for TAS was proposed by Nakayama and Hisano [9]. The proposed idea enabled runtime reconfiguration of the GCL to avoid reduction in bandwidth utilization irrespective of the burst size and the number of streams.

In case routing can be also decided, e.g., in layer 3, the joint routing and scheduling problem remains a challenge to be tackled. Falk et al. [10] presented a formulation in the ILP framework, which models the joint routing and scheduling problem for flows of periodic real-time transmissions in converged TSN networks. Network calculus-based flow routing and bandwidth allocation in IP-over-WDM architecture is also investigated to achieve the deterministic data delivery in metro-aggregation networks [11]. Krolikowski et al. [3] also focused on the joint routing and scheduling problem in large-scale deterministic networks using CSQF to maximize traffic acceptance for network planning and online flow admission. Joint routing and network resource allocation for the deterministic service function chaining (SFC) problem was also investigated in [12], where a novel deterministic SFC deployment algorithm and an SFC adjustment algorithm were proposed to ensure deterministic performance during the SFC lifetime.

To the best of our knowledge, limited work has been done on AI-based methods for DFRS, and the problem of deterministic latency (not minimizing the E2E latency) provisioning upon the mix-criticality flows has been solved. Therefore, we use DRL to solve the DFRS problem from this perspective, which is different from the abovementioned work.

III. System Model and Problem Formulation

A DN system refers to a network comprised of DN nodes (i.e., routers), whereby the packet forwarding delay inside a node is deterministic and known through a centralized flow scheduling

scheme. In a DN system, the delay induced by forwarding a packet comprises of following four parts:

1) propagation delay, which is decided by the physical distance between two nodes;
2) processing delay, which is related to the procedure of receiving the packets and sending them to the upper layers for routing and scheduling decision;
3) transmission delay, which is the time for putting the packet on the physical link;
4) queuing delay, which refers to the waiting time in the queue of the output port because of the accumulation of packets from different input ports to the same output port [13].

If the topology information (i.e., distance between any nodes and bandwidth of physical link) is given, the propagation, processing, and transmission delay can be assumed to be constant. Thus, to make the overall forwarding delay to be deterministic, the queuing delay should be determined properly in advance, ensuring the sum of delays along the path (E2E delay) equal to a constant and is within the latency requirements. All notations in this article are summarized in Table I.

TABLE I
NOTATIONS AND VARIABLES

Notations	Description
	Topology
$\mathcal{G}$	Substrate networks
$\mathcal{V}, \mathcal{E}$	Set of nodes and edges in substrate networks
u, v	Physical nodes in the network $\mathcal{G}$
e_i	Physical links in the network $\mathcal{G}$
d_{e_i}	Link delay of edge e_i
$\mathcal{T}$	Set of cycles
T, cap	Cycle duration and cycle capacity
	Service requests
$\mathcal{F}$	Set of DN flows
f_k	DN flow k
src_k, dst_k	Source and destination node of flow k
prd_k	Period of flow k
bw_k	Traffic load of flow k within one cycle
$D_k^{\max}$, $D_k^{\min}$	Maximal & minimal delay bound for flow k
$\mathcal{U}_k(t)$	Utility function of a SRT flow k with t
$\mathcal{H}_k$	If HRT flow k is scheduled successfully
prd_s	Least common multiple of periods of all flows
	Decision Variables
o_{k,e_i}	Transmission offset of flow k on edge e_i
e_i^k	Edge on which flow k is routed
t_i^k	Index of cycle of edge e_i on which the packets of flow k are transmitted
	DRL-related notations
s_t	Network state at time t
a_i^k	The ith subaction for flow k
$\mathcal{R}(a_i^k)$	Reward of the ith subaction for flow k
U_{e_i}	Link utilization rate of e_i
U_s	Global link usage of topology
$Q_{e_i}^t$	Whether cycle t on e_i is fully occupied
$I_{e_i}^t$	Cycle utilization rate of t on e_i
I_{e_i}	Overall cycle utilization rate of edge e_i
$P_{e_i}^t$	Traffic load in cycle t of edge e_i
n_d	Number of sub-actions of the dth dimension
$A_d(s_t, a_i^k)$	Advantage of the dth dimension
$V(s_t)$	Common state value
M	Action dimension
	System Parameters
α, β, η	Reward coefficients
γ	Discount factor
N, N_D	The number of total and time-sensitive queues within a node

A. CSQF-Enabled DN System

Initially, cycle queuing and forwarding [(CQF) [14], i.e., IEEE 802.1Qch] is proposed as a peristaltic shaper, which considers two queues on ports to be open and closed alternatively in a cyclic fashion. It divides the time into different cycles with an equal duration T. A packet sent from the precedent node in cycle c must be received during the same cycle in the subsequent node and then transmitted in cycle $c+1$. Although CQF can control well the delay over each hop (at most two cycles), the scalability of this mechanism is not enough since it only works well for small networks and assumes perfect synchronization between nodes.

To improve scalability and flexibility, the CSQF mechanism [15] has been devised as an emerging standard draft from the IETF DN working group as the evolution of the CQF mechanism. CSQF is proposed to delay packets with more queues and specify a certain cycle to transmit packets. Inside a CSQF-enabled router, N queues will be equipped in each output port and N_D queues out of N ($N_D \leq N$) queues are reserved for time-critical traffic, whereas the remaining noncritical (NC) queues are for best effort (BE) traffic. These N queues transmit packets in a round-robin fashion, that is, during each cycle, only one queue is active for emitting a packet to the physical link, the other $(N-1)$ inactive queues are closed and enqueue packets for future transmissions. Note that the number of packets that are enqueued in each inactive queue is related with the buffer size of each queue, and improper enqueuing will incur packet loss. The N_D time-sensitive queues are dedicated to the time-critical flows by resource reservation. The assignment of packets to specific queues actually decides their transmission cycle, and a packet can be delayed by at most $(N-1)$ cycles. This assignment can be determined by a centralized controller in advance, whereas the BE flows without critical timing requirements will not be scheduled in advance by the controller. When the packets of BE flows arrive at each node, they will be directly inserted into the $(N-N_D)$ NC queues, whose queuing delay is not controllable or deterministic. Note that unlike CQF, CSQF operates at layer 3 [15], as it allows to specify the routing and cycle scheduling of packets (e.g., with SR).

B. DN and Flow Model

Network topology in this article is modeled as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of nodes representing DN enabled routers. The nodes are connected with data links represented by the directed edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. If there exists a physical link between $u, v \in V$, then $(u, v), (v, u) \in \mathcal{E}$. Each edge $e_i = (u, v) \in \mathcal{E}$ induces a delay d_{e_i} that comprises its propagation delay, transmission delay, as well as processing delay. Time is partitioned into cycles of equal duration T, and $\mathcal{T}$ represents the set of cycles. One cycle is the minimal scheduling unit that

packets can be inserted into and defines the granularity of latency calculation.

A DN flow is defined as a periodic unicast traffic from a source node to a destination node. We denote the set of DN flows as $\mathcal{F}$ to be scheduled within the network $\mathcal{G}$. A DN flow $f_k \in \mathcal{F}$ is defined as a tuple $(\text{src}_k, \text{dst}_k, \text{prd}_k, \text{bw}_k, D_k^{\max}$, and $D_k^{\min})$, where the following holds.

1) src_k and dst_k represent the source and destination nodes of flow f_k.
2) prd_k denotes the period of flow f_k, which means the source node sends the packets every prd_k cycles.
3) bw_k is the total traffic that the source node emits in one cycle.
4) The delay experienced by packets should be larger than a minimum delay bound $D_k^{\min}$ and smaller than a maximum delay bound $D_k^{\max}$, as then the jitter does not exceed $(D_k^{\max} - D_k^{\min})$.

Since flows are featured with the different period prd_k, we define an overall scheduling cycle, which is referred to as hypercycle, so that all network behaviors are the same in each hypercycle. The hypercycle prd_s of all flows can be calculated as the *least common multiple* of the periods of all flows. Hence, we will discuss the flow scheduling problem in one hypercycle. Actually, the starting time of the hypercycles at different nodes may be not synchronized and there exists an offset between two nodes due to clock drift, which can be measured and known by the controller. For the sake of simplicity and without loss of generality, we assume there is no offset so that all hypercycles are aligned across the networks.

Furthermore, considering criticalities in terms of latency requirements, the DN flows can be further classified into: HRT flows, which have strict deadlines, and SRT flows whose quality of service (QoS) can be downgraded due to deadline violation. BE flows, which have no timing requirements, will be also considered in this article as background flows. Both HRT and SRT flows have the delay bounds $D_k^{\max}$ and $D_k^{\min}$. However, the HRT delay bound is *hard*, and if the delay bounds are violated, it may result in catastrophic consequences. The scheduling policy must guarantee that all HRT flows in the networks are transmitted within the delay bounds. The SRT deadline is *soft*, that is, the performance of the SRT flow will degrade if the delay bounds are missed. Similar to [16] and [17], a positive utility function is introduced to evaluate the performance of SRT flows, denoted with $\mathcal{U}_k(t)$, whereby t is the actual experienced E2E delay. If the packets of a SRT flow arrive within the delay bounds, the utility keeps on a predefined positive value (maximal value). The utility function decreases to zero with an actual E2E delay when it goes beyond the delay bounds, as specified by the definition of the utility function $\mathcal{U}_k(t)$.

C. DFRS: An Example

To ensure that no collision or congestion can happen, the controller needs to decide, for each packet, where and when it will be transmitted in each node, i.e., if a packet is sent in the first available cycle or delayed by one or more additional cycles before transmission.

In Fig. 1, we show an example of how a packet is propagated from node A to node D through nodes B and C. We assume that 1) the link delays of $d_{A,B}$, $d_{B,C}$, and $d_{C,D}$ are one cycle, two cycles, and one cycle, respectively; 2) the period of the flow of interest (FOI) is two cycles. Once the packets of FOI are sent from A, they are received at B in the next cycle (e.g., packet 1 is sent in cycle 1 at node A and received in cycle 2 at node B), since the link delay between nodes A and B is one cycle. Upon receiving packet 1 in cycle 2, the controller can decide to forward packet 1 in the next cycle (cycle 3). However, considering the high traffic load in cycle 3 of node B, it is better to choose to delay the packet forwarding by two cycles (i.e., CSQF offset), that is, packet 1 is forwarded in cycle 4. Then, it is received at cycle 6 due to two cycles delay between nodes B and C. The E2E delay of a packet is calculated as the number of cycles it costs along the path. For example, the E2E of packet 1 is seven cycles (cycles 1–8). What the controller should accomplish is, on the one hand, to ensure that the E2E delay of packets are within the delay bounds of this flow; on the other hand, to avoid the network congestion on certain cycles or edges.

D. DFRS: Multipath Case

For more efficient flow scheduling and higher load balancing, multipath TCP (MPTCP) technology [18], which allows multiple paths in a single TCP connection by spreading traffic data across multiple parallel subflows, has shown great advantages in emerging scenarios where heavy traffic needs to be transmitted. Different from the single-path flow scheduling in Fig. 1, flow splitting and multipath routing are considered in this scenario, where one single communication path is not sufficient to transmit the DN flow and multicommunication paths become needed. As shown in Fig. 2, we assume that a TCP connection f_1 is upcoming between Hosts 1 and 2, whose maximum and minimum deadlines are five and three cycles. If the scheduler fails to find a valid schedule along the single path, for example, there is not enough bandwidth for flow 1 within cycles 4–6 in node B, multipath scheduling will be applied to this flow by splitting it into multiple subflows evenly. For instance, there are two candidate routing paths $\{(A, B)\}$ and $\{(A, C), (C, B)\}$ that can be leveraged for two subflows, i.e., $f_{1.1}$ and $f_{1.2}$. Eventually, the E2E delay of these two subflows are four and five cycles separately, and packets of subflows will be reassembled at Host 2 without violating the deterministic delay requirement of flow f_1.

E. DFRS Modeling

For a flow f_k to be scheduled, the controller needs to determine a unique feasible scheduled path $\mathcal{P}$, i.e., a sequence of edges $(e_1, e_2, \ldots, e_n)$, where edge e_0 starts from src_k and edge e_n ends at dst_k. e_i and e_{i+1} are adjacent edges. We should have the constraints

$$e_0.\text{src} = \text{src}_k \tag{1}$$

$$e_n.\text{dst} = \text{dst}_k \tag{2}$$

$$e_i.\text{dst} = e_{i+1}.\text{src}. \tag{3}$$

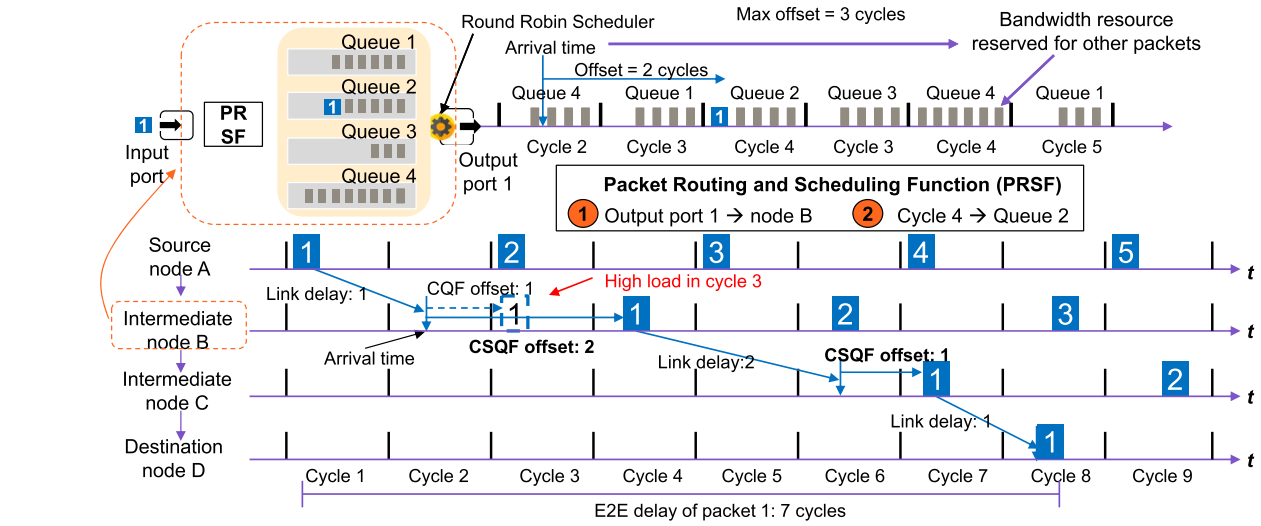


Fig. 1. CSQF-based cycle scheduling of a DN flow.

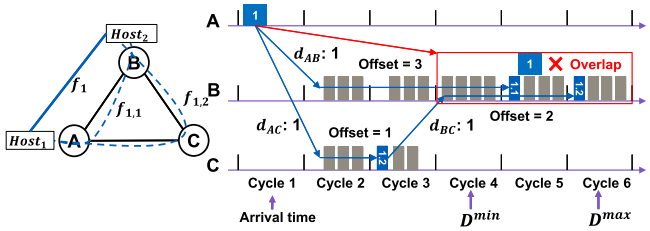


Fig. 2. CSQF-based multipath scheduling.

However, at each edge e_i, it is impossible to determine the transmission cycles for every packet of this target flow due to the high scheduling complexity. We define an integer variable o_{k,e_i} to represent the offset at edge e_i for all packets of flow f_k, that is, all packets of flow f_k arrived at edge e_i will be delayed by o_{k,e_i} cycles. If we assume that the first packet of flow f_k leaves the source node src_k at t_1^k, then it will arrive at the next node on the cycle $t_1 + d_{e_1}$, and will be transmitted again on cycle $t_1^k + d_{e_1} + o_{k,e_i}$, denoted by t_2^k. Therefore, the cycle determination for flow k can be represented by an integer sequence $(t_1^k, t_2^k, \ldots, t_n^k)$, where $t_i^k \in Z^+$ indicates the index of cycle that the first packet is supposed to be forwarded at the corresponding node e_i. The transmission cycles of the remaining packets can be calculated by $t_i^k + l * \text{prd}_k$, $l \in \{0, 1, \ldots, n\}$ easily.

Thus, the schedule $\mathcal{S}_k$ of a DN flow f_k from source node src_k to destination node dst_k can be denoted as $\{(e_1^k, t_1^k), (e_2^k, t_2^k), \ldots, (e_n^k, t_n^k)\}$. An $\mathcal{S}_k$ is valid for flow f_k if the following two conditions hold.

1) *E2E latency constraint:* The E2E delay of all packets of HRT flow f_k must not exceed the maximum and minimum E2E delay bounds $D_k^{\max}$ and $D_k^{\min}$

$$\mathbf{C1}: D_k^{\min} \leq t_n^k - t_1^k \leq D_k^{\max}. \tag{4}$$

2) *Cycle capacity constraint:* If the packets of flow f_k are decided to be transmitted at edge e_i at cycle t, denoted by x_{k,e_i}^t, the bandwidth capacity bw_k in the corresponding cycles will be reserved. Since the bandwidth capacities of cycles are shared among the scheduled flows, the traffic load at any edge e_i during any cycle t must not exceed its capacity cap, which is the value of cycle duration T multiplied by link data rate G. This condition is ensured by the constraint

$$C2: \sum_{f_k \in D} x_{k,e_i}^t * \text{bw}_k \leq \text{cap} \quad \forall e_i \in \mathcal{E} \quad \forall t \in \mathcal{T}. \tag{5}$$

F. Problem Formulation

The flow scheduling problem can be formulated as: given the network topology and DN flows, finding valid schedules (route and cycle allocation) for all flows so that all HRT flows are scheduled and the utility of SRT flows are maximized. We define the variable $\mathcal{H}_k$ to indicate if the HRT flow f_k is successfully scheduled: $\mathcal{H}_k = 1$ when HRT flow k is successfully scheduled, 0 otherwise. The DFRS problem can be defined as follows:

$$\max \sum_{f_k \in \mathcal{F}} \begin{cases} \mathcal{H}_k & \text{if } f_k \text{ is HRT flow} \\ \mathcal{U}_k & \text{if } f_k \text{ is SRT flow} \end{cases} \tag{6}$$

$$\text{s.t. } \mathbf{C1}, \mathbf{C2}.$$

G. MDP-Based Model

The learning process of reinforcement learning (RL) is usually modeled as a MDP, with the state space $\mathcal{S}$, the action space $\mathcal{A}$, and the reward $\mathcal{R}$ devised as follows.

1) State Space: A system state s_t represents all the information of the whole network that the RL agent can observe and use to generate a schedule $\mathcal{S}_k$ for a flow f_k. We denote network state s_t by extracting the network features from the following three aspects:

1) topology information,
2) flow information, and
3) network load information.

s_t can be divided into $s_t = \{s_{t,e_1}, s_{t,e_2}, \ldots, s_{t,e_i}, e_i \in \mathcal{E}\}$ and each s_{t,e_i} consists of.

1) If the edge e_i is adjacent to the edge of the former action or to the source node src_k.
2) The distance between this edge and destination node dst_k.
3) The difference between the current selected cycle and $d_k^{\min}$, (i.e., minimum delay requirement-passed delay).
4) The difference between the current selected cycle and $d_k^{\max}$.
5) The traffic load of this edge, which is denoted as the ratio of the number of available cycles to the total number of cycles. Generally, choosing an edge with lower network load can maintain load balancing and avoid bottleneck links.
6) The list of cycle load (in percentage). The state s_t will be updated each time after the agent selects a subaction.

2) Action Space: Deep-DFS improves the scalability by dividing the schedule of a flow into a set of sub-actions. That is, the schedule $\mathcal{S}_k = \{a_1^k, a_2^k, \ldots, a_n^k\} = \{(e_1^k, t_1^k), (e_2^k, t_2^k), \ldots, (e_n^k, t_n^k)\}$, where $a_i^k = (e_i^k, t_i^k)$, of flow k is derived from a sequence of edges and cycles. Specifically, each $a_i^k = (e_i^k, t_i^k)$ acts as a subaction, where e_i^k denotes the edge that a flow needs to go through, and t_i^k represents the cycle that packets are transmitted on e_i^k. A valid path should satisfy the following conditions: 1) the edges in subactions should connect in head-to-tail; 2) the constraint $t_{i+1}^k > t_i^k + d_{e_i}$ should be kept, where d_{e_i} is the link delay of e_i, to ensure a valid timing. Combined with constraint (4) and (5), a valid schedule for flow f_k should meet these four constraints.

3) Reward Function: After receiving a subaction a_i^k, the agent will obtain a reward value $\mathcal{R}(a_i^k)$ from the environment based on the effect that this subaction causes.

The reward of a subaction a_i^k will comprise of two parts: 1) how much congestion it brings to the network; 2) whether this subaction will finalize a complete schedule of a flow, which is also valid.

We define the link utilization rate U_{e_i} as the number of cycles, which are not available for flows divided by the number of all cycles on link e_i

$$U_{e_i} = \frac{\sum_{t\in\mathcal{T}} Q_{e_i}^t}{|\mathcal{T}|} \tag{7}$$

where $Q_{e_i}^t$ represents whether cycle t on e_i is fully occupied, and $|\mathcal{T}|$ and $|\mathcal{E}|$ are the total number of cycles considered in one hypercycle and the number of edges in the topology. Furthermore, the global bandwidth utilization ratio U_s over all cycles and edges in the network is defined as follows:

$$U_s = \frac{\sum_{e_i\in\mathcal{E}}\sum_{t\in\mathcal{T}} Q_{e_i}^t}{|\mathcal{T}||\mathcal{E}|}. \tag{8}$$

Besides evaluating the impact of a selected subaction on the link utilization rate, the cycle utilization rate $I_{e_i}^t$ should be also considered, which is defined as follows:

$$I_{e_i}^t = \frac{\sum_{l=0}^{\frac{\text{prd}_s}{\text{prd}_k}-1} P_{e_i}^{t+l*\text{prd}_k}}{\frac{\text{prd}_s}{\text{prd}_k} * \text{cap}} \tag{9}$$

where $P_{e_i}^t$ denotes traffic load in cycle t of edge e_i. Thus, the overall cycle utilization rate of edge e_i can be defined as follows:

$$I_{e_i} = \frac{\sum_{t\in\mathcal{T}} P_{e_i}^t}{|\mathcal{T}|\text{cap}}. \tag{10}$$

In order to make the training converge fast, we use α, β, η to adjust the weight of each part making the reward value within $(-1, 1)$, and then, the reward of the subaction a_i is denoted as follows:

$$\mathcal{R}\left(a_i^k\right) = \alpha\left(U_s - U_{e_i}\right) + \beta\left(I_{e_i}^t - I_{e_i}\right). \tag{11}$$

Note that if the usage U_{e_i} is larger than the global usage U_s after mapping the flow to edge e_i, it means a negative reward for this subaction a_i. The same applies for cycle usage I_{e_i} and $I_{e_i}^t$.

The second part takes effect only if the subaction a_i^k is the last edge of a valid path for a flow. If the agent finalizes the scheduling of a flow, $\mathcal{H}_k$ or $\mathcal{U}_k$ will be calculated according to the flow types.

After selecting the last action a_n^k of a valid route, we will check if this flow is scheduled successfully (i.e., if the latency, capacity and routing requirement are all satisfied), and then, the rewards for the subactions $\mathcal{R}(a_1^k), \mathcal{R}(a_2^k), \ldots, \mathcal{R}(a_n^k)$ will be updated by adding an extra reward for the second part in a decayed way. Intuitively, earlier subactions have a larger exploration space, and thus pose less impact on a valid schedule, whereas the latter subactions are more significant for constituting a valid schedule

$$\hat{\mathcal{R}}\left(a_i^k\right) = \mathcal{R}\left(a_i^k\right) + \eta\{\mathcal{H}_k, \mathcal{U}_k\} \times \frac{i}{n} \quad \forall i \in \{1, \ldots, n\}. \tag{12}$$

H. Optimization Formulation

This article aims to obtain the optimal DFRS policy, denoted by π^*, to maximize the long-term rewards of mapping flows into networks. The DFRS problem can be transferred to the optimization problem which maximizes the expected future discounted rewards as follows:

$$\max_{\pi} \mathbb{E}\left[\sum_{f_k\in\mathcal{F}}\sum_{i=0}^{n} \gamma^i \mathcal{R}(a_i^k)\right] \tag{13}$$

where $\mathcal{R}(a_i^k)$ is the reshaped reward under the policy π from (12), and the discount factor γ indicates how much the current rewards are valuable than later rewards.

To solve the optimization problem in (13), we propose a BDQN-based DFRS algorithm, which will evaluate each action dimension (i.e., edge and cycle selection) separately, and also use action masking to improve the training efficiency.

IV. Branching Dueling Q-Network-Based DFRS

To facilitate the learning process for a MDP problem, deep Q-Network (DQN)-based methods are widely leveraged in the literature. For example, dueling DQN is proposed to eliminate overestimation in the learning process and improve the performance of the double deep Q-network (double DQN) algorithm [19], [20]. By applying a primary neural network Q_{net} as a nonlinear function approximator to select an action, and using a target neural network Q_{target} to estimate the target Q-value of

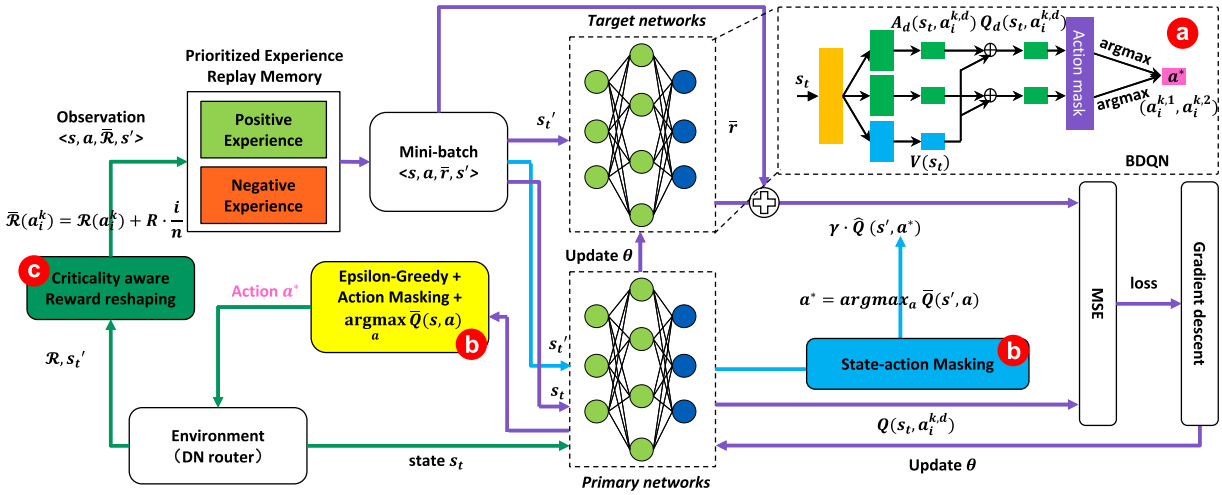


Fig. 3. Branching DQN-based learning process.

the taken action, double DQN stabilizes the training process of the RL agent. Dueling DQN further improves double DQN by using two separate neural networks to estimate the state value and the action advantage, and then, the state values and the action advantages are aggregated at the output layer. By doing this, dueling DQN can perform more robust estimations on state values, which lead to significant improvements on convergence rate and the stability of the learning process.

However, in order to solve the DFRS problem defined in this article using the dueling DQN method, several challenges still remain to be tackled.

1) Unlike other simple scheduling tasks where the action space is featured with only single dimension, the action space of DFRS problem is with two dimensions, i.e., edge and cycle selection, which are independent from each other. However, these two dimensions are synergistic when scheduling DN flows for deterministic performance. The edge selection should consider the cycle utilization of network and cycle selection also depends on the selected edge/path, which brings a huge challenge to deterministic flow scheduling.
2) To avoid a large amount of invalid actions for scheduling a DN flow and improve learning efficiency, the size of the action space, which is proportional with the amount of the edges in the topology and cycles considered in a hypercycle, can be further reduced.
3) Specialized rewards for flows with different criticalities should be designed so that the DN flows are scheduled with different priorities.

Therefore, as shown in Fig. 3, three approaches or techniques are proposed in this section to respond to the challenges mentioned above. Generally, two neural networks are employed, the Primary network for selecting an action, and the Target network for estimating the target Q-value of the taken action. The parameter θ of target networks will be updated from primary networks every certain number of iterations. The experience replay technique is also adopted to stabilize the learning process. In addition, to improve the learning efficiency and accuracy, we carry out the following.

1) We incorporate the advances of action branching with dueling deep Q-network (dueling DQN) [21], which is referred to as a BDQN, to solve the action selection with multiple dimensions.
2) We use the action masking to block a large amount of invalid actions. A delay-aware masking method is designed to reduce the size of the action space while ensuring enough exploration space.
3) We design a criticality-aware reward function to entitle different priorities for DN flows with different types, e.g., high priority for HRT flows.

A. Branching Dueling Q-Network

The action branching methods proposed in [22] improve the conventional deep Q-network to solve MDP with multidimensional discrete action space. The main idea is to evaluate the individual action dimension $a_i^{k,d} \in \mathcal{A}_d, d \in \{1, \ldots, M\}$, e.g., edge and cycle selection in the DFRS problem, while keeping a common state-value estimator between multiple dimensions. Each dimension has a certain degree of autonomy. The structure of BDQN is illustrated in Fig. 3(a), the BDQN further splits the advantage branch into two advantage branches based on dueling DQN, while keeping a shared representation of the input state. Specifically, the advantage branches correspond to the two dimensions of action in this study, i.e., $a_i^{k,1} = e_i^k, a_i^{k,2} = t_i^k$, each dimension has n_d subactions. For example, the n_d of the edge dimension is $|\mathcal{E}|$. As shown in Fig. 3(a), a network state s_t is input into a shared neural network (yellow block), which will then compute a latent representation that is used for the evaluation of the state value (blue block) $V(s_t)$ and the factorized (state dependent) action advantages (green block) $A_d(s_t, a_i^{k,d})$ on the subsequent independent branches. The dimensions of action advantages and state value are aligned, i.e., $\max\{n_1, \ldots, n_M\}$. Then, state value $V(s_t)$ and the factorized advantages are combined to output the Q-values for each action dimension. Note that in order to filter the Q-values for the valid subaction of each action dimension, action masking is applied here to accelerate the learning processing. Finally, the subactions with maximal Q-values of

each action dimension are selected for the generation of a joint action tuple.

The advantage of each dimension, i.e., $A_d(s_t, a_i^{k,d})$ is trained with the common state value $V(s_t)$, and then, the Q-value of each dimension $Q_d(s_t, a_i^{k,d})$ is obtained by aggregating the value branch and the corresponding advantage branch as follows:

$$Q_d\left(s_t, a_i^{k,d}\right) = V(s_t) + \left(A_d\left(s_t, a_i^{k,d}\right) - \frac{1}{n_d}\sum_{\hat{a}_i^{k,d}\in\mathcal{A}_d} A_d\left(s_t, \hat{a}_i^{k,d}\right)\right). \quad (14)$$

Then, the action selection follows the ϵ-greedy policy, that is, select a random action with probability ϵ and with probability $(1-\epsilon)$ to select

$$a = \left\{\arg\max_{a_i^{k,1}} Q_1\left(s_t, a_i^{k,1}; \boldsymbol{\theta}\right), \ldots, \arg\max_{a_i^{k,M}} Q_d\left(s_t, a_i^{k,M}; \boldsymbol{\theta}\right)\right\}. \quad (15)$$

The TD-target of BDQN is similar with that in dueling DQN to avoid maximization bias, but it is derived by averaging across all dimensions of the action

$$y = \hat{\mathcal{R}}\left(a_i^k\right) + \delta\frac{1}{M}\sum_d \hat{Q}_d\left(s_t', \arg\max_{\hat{a}_i^{k,d}\in\mathcal{A}_d} Q\prime_d\left(s_t', \hat{a}_i^{k,d}\right)\right). \quad (16)$$

We use mean square error (mse) to update the parameter $\boldsymbol{\theta}$ by the gradient descent method.

B. Action Masking

At the early stage of training, the agent learns to find a valid schedule of a flow in an exploring way. The subaction is selected by the agent for the action space $|\mathcal{E}| \times |\mathcal{T}|$, most edges and cycles in the action selected at this stage are not valid for constituting a feasible path, which will slow down the learning efficiency. To avoid the sparse rewards induced by the invalid actions in the training processing, action masking is proposed in this section to block the actions that are obviously invalid to improve the learning efficiency.

Action masking will be applied in two phases of the whole training processing: 1) the action selection phase and 2) the Q value updating phase. We maintain binary lists $[a_i]$, $[c_j]$, where $i \in \mathcal{E}$ and $j \in \mathcal{T}$ to filter the invalid actions each time the agent selects a subaction. $a_i = 1$ if ith edge is adjacent to the edge selected by the former action, otherwise $a_i = 0$. When selecting valid cycles along the path, two aspects should be considered: 1) maximum delay (offset) in each switch and 2) residual available latency budget. Since the queues in a router transmit the packets in a round-robin way, which means each queue transmits packets every N cycles, the packets will be delayed at most $(N-1)$ cycles in a switch, that is $c_j = 1$ for $j \in (t, t+N-1)$ should be satisfied, where t is the cycle selected for the former subaction.

Besides the constraint of maximum offset in one node, the cycle selection should also satisfy the E2E latency requirement. Therefore, each time the agent selects an action, the actual latency that this flow consumes should not exceed the maximum latency bound $D_k^{\max}$. That is, $c_j = 1$ for $j \in (t, D_k^{\max})$; otherwise, $c_j = 0$. The list $[a_i]$, $[c_j]$ is recalculated and multiplied by original Q values each time when making a subaction decision. This narrows down the scope of action selection, the agent chooses an edge and cycle from valid actions that have the highest Q value, which can produce more positive experience and improve the sample efficiency in the training process.

Action masking is also applied in the process of Q-value updating. During the learning period, to avoid the overestimation of the actual Q-value, action selection is based on the Q value of policy network $Q(s,a)$ in the TD-error calculation as (16). In this section, we modify the $Q_d(s', \hat{a}_i^{k,d})$ by multiplying it with $[a_i]$, $[c_j]$, that is, $Q'_d(s', \hat{a}_i^{k,d})$.

TABLE II
FLOW TYPES WITH DIFFERENT CRITICALITIES

Flow type	Size	Period	Delay Bounds
$\mathcal{F}^{\text{HRT}}$	{1,2,4} DUs	{2,4,8,16} cycles	{(8,10),(18,20)} cycles
$\mathcal{F}^{\text{SRT}}$	{1,2,4} DUs	{2,4,8,16} cycles	{(6,8,10,12), (16,18,20,22)} cycles
$\mathcal{F}^{\text{BE}}$	{2} DUs	{2} cycles	—-

C. Criticality-Aware Reward Function

Although the controller can schedule the real-time flows by assigning the cycles for packet transmission along the path with the CSQF mechanism, it cannot always guarantee all real-time flows are scheduled successfully because of the resource competition. Therefore, deep-DFS should learn the criticalities of different DN flows, so that all HRT flows are scheduled successfully and the total utility for SRT flows is maximized. Besides $\mathcal{H}_k$, which can represent if an HRT flow is scheduled, $\mathcal{U}_k$ can be defined by a linear function, starting at a maximum utility of 1, linearly decreasing after breaking the delay bounds, and reaching a zero utility at a certain value, e.g., $\mathcal{U}_k = 1$, when $8 < t < 10, \mathcal{U}_k = 0$ when $t < 6$ or $t > 12$. Therefore, the soft delay bound of an SRT flow is denoted as (6,8,10,12).

V. EVALUATION

A. Evaluation Settings

By conducting simulation experiments of deep-DFS, we demonstrate the effectiveness of deep-DFS in maximizing the number of scheduled flows under different network scales as well as flow distributions by comparing it with two benchmark methods.

We evaluate the performance of deep-DFS on a ladder network topology introduced in an Ethernet consist network [23], which is an international standard of train communication network. In this article, the size of the ladder topology is varied from 6 to 10 nodes. Besides the network topology, the DN flows for training and evaluation are both generated as per Table II with the probabilities of 40%, 40%, and 20% on the HRT, SRT, and BE types, respectively. Note that the flows of $\mathcal{F}^{\text{BE}}$ are considered as background traffic which has no deadlines. Within each flow, src_k and dst_k are randomly selected from all nodes

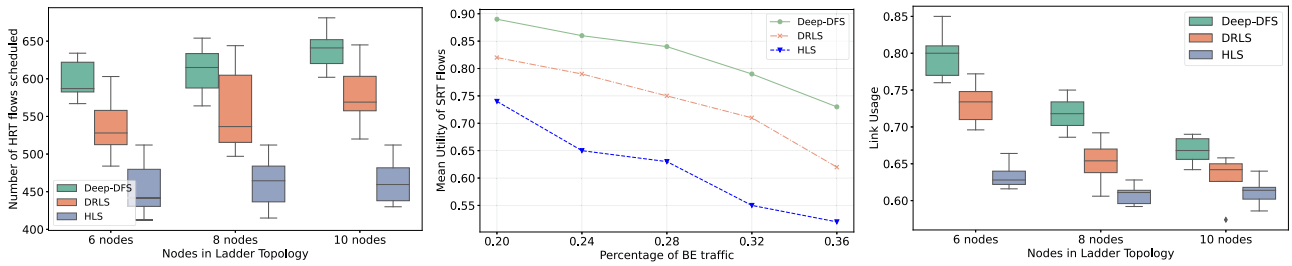


Fig. 4. (a) Number of HRT flows scheduled. (b) Utility of SRT flows. (c) Link Usage with different nodes in ladder topology.

in the networks. The packet length (in data units, DUs), period (in cycles), and delay bounds (in cycles) are selected randomly from the set in Table II. In addition, we also set the data rate of all physical links to 1 Gbit/s, and the capacity of each cycle is 100 DUs. The hypercycle is 16 according to periods of all flows.

For the configuration of the BDQN, the parameters are employed based on the common settings for designing neural networks [19], i.e., two fully connected hidden layers are together deployed with input and output layers. The size of the hidden layers is 128, and the size of the output layer is 2, which represents the index of the selected edge and cycle. The minibatch size is 32, and the discount factor γ is set to 0.5. The initial value of ϵ is 1.0 and decays to its final value 0.01. The agent will be trained and evaluated on the same topology but with different flows, separately.

B. Baseline Methods Compared

To evaluate the effectiveness of deep-DFS, we compare it with the following baseline methods.

1) *DRLS:* The DRL-based TTEthernet scheduler [6] outputs edge action step by step to constitute a complete flow schedule, but use a heuristic method to select the earliest available cycles with low degree.
2) *Heuristic list scheduler (HLS):* HLS selects [5] a time slot that leads to the minimum E2E delay on the shortest routing path between the source node and the destination node of a flow. If this minimum delay exceeds the deadline of f_k, the scheduler fails to schedule this flow.

C. Incremental Scheduling Scenario

In this scenario, we randomly generate the flow one by one and insert the flow to the network incrementally. If the agent fails to schedule an HRT flow then it stops, and then, we compare the maximum number of successfully scheduled HRT flows and the utility of the SRT flows of each solution.

As shown in Fig. 4(a), deep-DFS can schedule more HRT flows than the other two methods in general, specifically, 14.8% more on average than DRLS and 32.1% more on average than HLS in ladder networks. Since HLS always selects the first available time slot to transmit the packets on the shortest path, it will derive the minimum latency for all flows regardless of the flow criticalities and their delay bounds. Therefore, HLS will saturate the cycles and edges soon, and increase the probability of flow blocking by some fully occupied cycles. DRLS takes into account the cycle usage on the edge, it avoids selecting the cycles with high degree in order to save more bandwidth for the flows with different periods. However, DRLS still tries to select the first available cycle for packet forwarding and minimizes the E2E delay of the flow, which is in conflict with the long-term objective of maximizing the number of flows scheduled in this system. To this end, deep-DFS redesigns the network state representation and reward function to make delay-aware decisions on edge and cycle selection, so that the HRT flows are prioritized and no bandwidth resources are wasted on minimizing the E2E delay. When the size of the ladder topology becomes larger, deep-DFS schedules more flows (39.1% more than HLS on average) in the ladder topology with ten nodes than with six nodes (29.3% more than HLS on average). This is because deep-DFS has more exploration space in a larger topology whereas HLS only selects the shortest paths to route the flows regardless of topology size.

We also evaluate the average utility of SHR flows with the percentage of BE traffic. We set the probability of generating BE traffic from 0.2 to 0.36, with HRT and SRT flows are generated with the same probability. As we can see in Fig. 4(b), it is obvious that the average utility will decrease with more BE traffic inserted in the network. With the increasing BE traffic, the network bottleneck will come earlier and the utility of SRH flows in the HLS case will decrease a little faster than the other two methods due to the selection of shortest paths, as discussed previously. The link and cycle usages are also shown in Figs. 4(c) and 5(a). The results show that although the HLS method will lead to more cycles with high traffic load ($\geq 60\%$), the link usage induced by HLS is lower than the that of deep-DFS, which is not intuitional. This is because, on the one hand, the resources are exhausted in earlier cycles by minimizing the E2E delay with HLS, although HLS also stops to schedule flows earlier than deep-DFS. That makes the overall link usage of HSL lower than deep-DFS by 21.3% on average in the ladder topology. We also find that the link usage decreases slightly with more nodes in the ladder topology for deep-DFS, as it prefers to choose a longer route to balance the link load and avoid the bottleneck.

D. Multipath Scheduling Scenario

As shown in Fig. 6, to implement the multipath scheduling with a DRL solution, flow splitting module is needed in the DRL agent. The flow splitting and tagging function and the flow recovery function should be also deployed to map the multipath selection. Upon receiving the request information of flow f_k from the host, the DRL agent calculates the action for this flow.

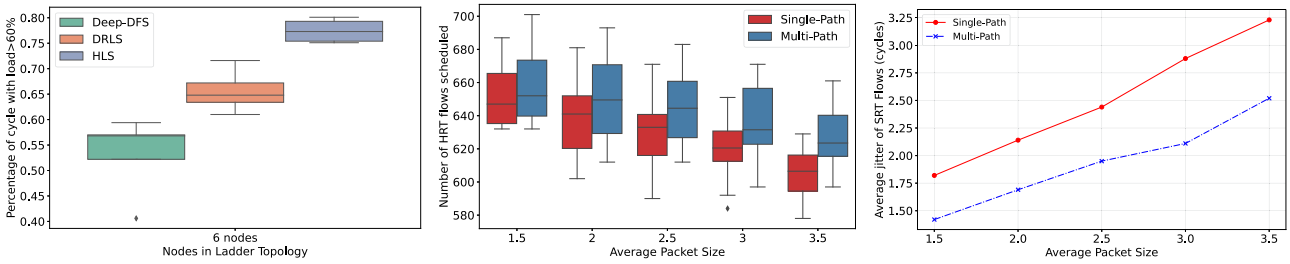


Fig. 5. (a) Percentage of cycles with traffic load over 60%. (b) Number of HRT flows scheduled under single and multipath scheduling. (c) Average jitter of SRT flows under single and multipath scheduling.

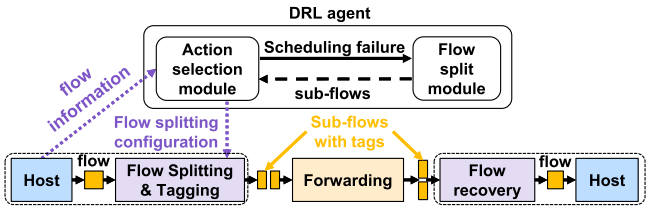


Fig. 6. DRL-based flow splitting and multipath scheduling.

If the agent fails to find a valid schedule for flow f_k, the flow split module modifies the request information by dividing the size of the flow into $1/q$, e.g., 1/2 while keeping other requirements unchanged. Then, the q subflows are fed into the action selection module again. If they are scheduled successfully, in other words, the q subflows are scheduled with different paths while the delay requirements of all subflows are satisfied, the corresponding flow splitting information will be tagged on the packets of this flow, and they will be reassembled in the destination host. Once the agent fails to schedule one of the q subflows, it stops.

We set the topology with ten nodes and generate the flows with an average packet size (in DUs) from 1.5–3.5. We also assume that $q = 2$ and 1 DU is the minimal transmission unit that cannot be split any further in this case. From Fig. 5(b), we can observe that the number of scheduled HRT flows decreases with the increase of average packet size, as high-size packets consume more bandwidth resources. However, compared with a single path schedule, multipath scheduling has better performance in finding valid schedules for HRT flows. In addition, the number of scheduled flows in multipath scheduling decreases more slowly than that of single path scheduling for the reason that the HRT flows have more chance to be scheduled after they are split into multiple subflows. We also evaluate the performance of multipath scheduling in terms of the jitter of the SRT flows, as shown in Fig. 5(c). As there is no hard deadline for the SRT flows, the jitter of the SRT flows is usually higher than that of the HRT flows. Because once an HRT flow f_k is scheduled, the delay jitter of this flow ought to be within $(D_k^{\max} - D_k^{\min})$. We find that multipath scheduling also outperforms single path scheduling in terms of jitter performance. Furthermore, the jitters of the SRT flows with multipath scheduling can be well controlled within three cycles on average, whereas the jitters with single path scheduling are much larger, due to the more severe competition for resources in case of only one transmission path.

VI. Conclusion

In this article, we proposed a deep-DFS to solve the scheduling problem of DNflows with multiple criticalities. We leverage deep-DFS to determine the routing and cycle selection for DN flows with the CSQF mechanism, where 1) the timeline is divided into multiple cycles with equal duration and 2) the controller can specify the cycles for packet forwarding so that the E2E delay of DN flows can be controlled effectively. To make the proposed Deep-DFS schedule the flows with multiple criticalities, several technologies were proposed to increase scalability and performance. Compared with the other AI-based and heuristic-based methods, deep-DFS can increase the scheduled flows by 14.8% and 32.1%, respectively. However, it is worth noting that the proposed centralized scheduling approach is merely suitable for scenarios with small network scales, e.g., within a factory network. If the network scale is large, the round-trip time of the signaling between the source node of a flow and the controller becomes also too large, which makes no sense to a time-sensitive flow. Furthermore, runtime reconfiguration is also a challenge for a centralized controller. Large-scale deterministic flow scheduling requires a disparate solution, e.g., distributed learning and distributed scheduling approaches, which can facilitate the learning process and network configuration locally. Therefore, the way to design a distributed learning architecture and train distributed learning agents for flow scheduling with SR efficiently will be considered in our future work.

References

[1] M. Wollschlaeger, T. Sauter, and J. Jasperneite, "The future of industrial communication: Automation networks in the era of the Internet of Things and industry 4.0," *IEEE Ind. Electron. Mag.*, vol. 11, no. 1, pp. 17–27, Mar. 2017.

[2] H. Yu, T. Taleb, and J. Zhang, "Deterministic service function chaining over beyond 5G edge fabric," in *Proc. IEEE Glob. Commun. Conf.*, 2021, pp. 1–6.

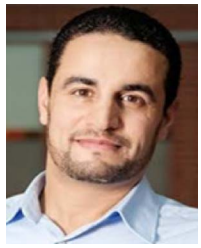
[3] J. Krolikowski et al., "Joint routing and scheduling for large-scale deterministic ip networks," *Comput. Commun.*, vol. 165, pp. 33–42, 2021.

[4] N. Wang, Q. Yu, H. Wan, X. Song, and X. Zhao, "Adaptive scheduling for multicluster time-triggered train communication networks," *IEEE Trans. Ind. Informat.*, vol. 15, no. 2, pp. 1120–1130, Feb. 2019.

[5] M. Pahlevan, N. Tabassam, and R. Obermaisser, "Heuristic list scheduler for time triggered traffic in time sensitive networks," *ACM Sigbed Rev.*, vol. 16, no. 1, pp. 15–20, 2019.
[6] C. Zhong, H. Jia, H. Wan, and X. Zhao, "Drls: A deep reinforcement learning based scheduler for time-triggered ethernet," in *Proc. IEEE Int. Conf. Comput. Commun. Netw.*, 2021, pp. 1–11.
[7] D. Hellmanns, J. Falk, A. Glavackij, R. Hummen, S. Kehrer, and F. Dürr, "On the performance of stream-based, class-based time-aware shaping and frame preemption in TSN," in *Proc. IEEE Int. Conf. Ind. Technol.*, 2020, pp. 298–303..
[8] N. Reusch, L. Zhao, S. S. Craciunas, P. Pop, "Window-based schedule synthesis for industrial IEEE 802.1 Qbv TSN networks," in *Proc. IEEE 16th Int. Conf. Factory Commun. Syst.*, 2020, pp. 1–4.
[9] Y. Nakayama and D. Hisano, "Multi-stream gate control of time aware shaper for high link utilization," in *Proc. IEEE Int. Conf. Commun. Workshops*, 2021, pp. 1–6.
[10] J. Falk, F. Dürr, and K. Rothermel, "Exploring practical limitations of joint routing and scheduling for tsn with ilp," in *Proc. IEEE 24th Int. Conf. Embedded Real-Time Comput. Syst. Appl.*, 2018, pp. 136–146.
[11] H. Yu, T. Taleb, J. Zhang, and H. Wang, "Deterministic latency bounded network slice deployment in ip-over-WDM based metro-aggregation networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 2, pp. 596–607, Mar./Apr. 2022.
[12] H. Yu, T. Taleb, and J. Zhang, "Deterministic latency/jitter-aware service function chaining over beyond 5G edge fabric," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 2148–2162, Sep. 2022.
[13] A. N. Abbou, T. Taleb, and J. Song, "Towards SDN-based deterministic networking: Deterministic E2E delay case," in *Proc. IEEE Glob. Commun. Conf.*, 2021, pp. 1–6.
[14] *IEEE standard for local and metropolitan area networks–bridges and bridged networks–amendment 29: Cyclic queuing and forwarding*, IEEE 802.1Qch-2017, Jun. 2017.
[15] M. Chen, X. Geng, and Z. Li, "Segment routing (SR) based bounded latency," *Internet Engineering Task Force, Internet-Draft draft-chendetnet-sr-based-bounded-latency-00*, 2018.
[16] V. Gavriluţ and P. Pop, "Traffic class assignment for mixed-criticality frames in ttethernet," *ACM Sigbed Rev.*, vol. 13, no. 4, pp. 31–36, 2016.
[17] G. Buttazzo, G. Lipari, L. Abeni, and M. Caccamo, *Soft Real-Time Systems*. Berlin, Germany: Springer, 2005.
[18] A. Ford, C. Raiciu, M. Handley, and O. Bonaventure, "TCP extensions for multipath operation with multiple addresses," Internet Eng. Task Force, Fremont, CA, USA, Tech. Rep. RFC 6824, 2013.
[19] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
[20] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
[21] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
[22] A. Tavakoli, F. Pardo, and P. Kormushev, "Action branching architectures for deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell., 30th Innov. Appl. Artif. Intell. Conf., 8th AAAI Symp. Educ. Adv. Artif. Intell.*, 2018, pp. 4131–4138.
[23] I. E. Commission et al., "Electronic railway equipment–train communication network (TCN)–part 3-4 (s): Ethernet consist network (ECN)," *Int. Electrotechnical Commission*, pp. 61375–3, 2014.

Hao Yu received the B.S. and Ph.D. degrees in communication engineering from the Beijing University of Posts and Telecommunications, Beijing, China, in 2015 and 2020.

He was also a Joint-Supervised Ph.D. Student with the Politecnico di Milano, Milano, Italy. He is currently a Postdoctoral Researcher with the Center of Wireless Communications, Oulu University, Oulu, Finland. His research interests include intelligent edge network, time sensitive networks, and 6G deterministic networking.



Tarik Taleb (Senior Member, IEEE) received the B.E. degree in information engineering with distinction, M.Sc. and Ph.D. degrees in information sciences from Tohoku University, Sendai, Japan, in 2001, 2003, and 2005, respectively.

He is currently a Professor with the Center of Wireless Communications, University of Oulu, Oulu, Finland. He is the founder and Director of the MOSA!C Lab (www.mosaic-lab.org), Espoo, Finland. Between October 2014 and December 2021, he was a Professor with the School of Electrical Engineering, Aalto University, Espoo, Finland. He was a Senior Researcher and 3GPP Standards Expert, NEC Europe Ltd, Heidelberg, Germany. Before joining NEC and till March 2009, he was an Assistant Professor with the Graduate School of Information Sciences, Tohoku University, Sendai, Japan, in a lab fully funded by KDDI, Tokyo, Japan. From October 2005 till March 2006, he was Research Fellow with the Intelligent Cosmos Research Institute, Sendai, Japan. His research interests include telco cloud, network softwarization and network slicing, AI-based software defined security, immersive communications, mobile multimedia streaming, and next generation mobile networking. He has been also directly engaged in the development and standardization of the Evolved Packet System as a Member of 3GPP's System Architecture working group 2.

Dr. Taleb was on the IEEE Communications Society Standardization Program Development Board. He was the General Chair of the 2019 edition of the IEEE Wireless Communications and Networking Conference (WCNC'19) held in Marrakech, Morocco. He was the Guest Editor in Chief of the IEEE JSAC Series on Network Softwarization and Enablers. He was on the editorial board of the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, IEEE WIRELESS COMMUNICATIONS MAGAZINE, IEEE JOURNAL ON INTERNET OF THINGS, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE Communications Surveys and Tutorials, and a number of Wiley journals. Till December 2016, he was the Chair of the Wireless Communications Technical Committee, the largest in IEEE ComSoC. He was the recipient of the 2021 IEEE ComSoc Wireless Communications Technical Committee Recognition Award December 2021, 2017 IEEE ComSoc Communications Software Technical Achievement Award, December 2017 for his outstanding contributions to network softwarization. He was also the (co-) recipient of the 2017 IEEE Communications Society Fred W. Ellersick Prize, May 2017, and many other awards from Japan. Some of his research works have been also awarded best paper awards at prestigious IEEE-flagged conferences.

Jiawei Zhang received the Ph.D. degree in telecommunication engineering from the State Key Laboratory of Information Photonics and Optical Communications, Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2014.

He is currently an Associate Professor with BUPT. He has authored and coauthored more than 30 OFC/ECOC papers and top journal papers in optical communication and networks. His research interests include the collaboration of optical networks with IP, wireless and cloud/edge, currently with an emphasis on the advanced technologies for providing deterministic connections for future network applications.

Dr. Zhang was on the Technical Program Committees for the IEEE DRCN 2018–2020, IEEE ICNC 2017-2018, ACP2020, and for the Workshop on Cloud Computing Systems, Networks and Applications at the IEEE GLOBECOM 2014–2016, ICC 2015–2016, and INFOCOM 2017–2018 conferences. He was the Guest Editor of the special issue on Resilience in Future 5G Photonic Networks of *Photonic Network Communications* journal (Springer).